

Compression d'images par factorisation SVD

9 mars 2026

Ce travail pratique a pour but d'implémenter un algorithme de compression d'images avec pertes. Cet algorithme reposera sur une méthode de factorisation matricielle, la factorisation SVD. Si dans un premier temps nous implémenterons des fonctions utilitaires ayant pour but de manipuler des matrices de Householder ; nous implémenterons ensuite un algorithme direct pour transformer une matrice A en une matrice bidiagonale BD . Par la suite, nous verrons comment transformer cette matrice BD en une matrice diagonale S de façon itérative pour finalement appliquer toutes ces transformations à l'algorithme de factorisation SVD.

Transformation de Householder

Dans cette partie, nous allons implémenter le produit d'une matrice de Householder par un ensemble de vecteurs. Notre objectif sera d'obtenir des algorithmes de complexité linéaire. Si à première vue nous sommes tentés de calculer la matrice de Householder pour appliquer la transformation, cette méthode ne nous permet pas d'obtenir un algorithme en complexité linéaire. Pour cela, il suffit de remarquer le fait suivant :

Soit H une matrice de Householder de taille n ; on se donne un vecteur $x \in \mathbb{R}^n$. On a alors :

$$\begin{aligned} Hx &= (I_n - 2NN^t)x, \text{ où } N \text{ est un vecteur unitaire dirigé par } V - U \\ &= x - 2N(N^tx) \\ &= x - 2N\langle N | x \rangle \end{aligned}$$

En effet, nous obtenons seulement 2 opérations linéaires : un produit scalaire et la différence de deux vecteurs. Nous avons donc là un algorithme dont la complexité est linéaire. Par la suite, nous avons aussi pu étendre cette idée au produit d'une matrice de Householder par une autre matrice pour obtenir un algorithme utile pour la suite.

Mise sous forme bidiagonale

L'objectif de cette nouvelle partie est de transformer toute matrice rectangle de dimension $n \times m$ en matrice bidiagonale supérieure. Le principe de cet algorithme est, à tout instant i , d'utiliser 2 transformations QR pour faire apparaître les 0 nécessaires sur la colonne i et sur la ligne i . Après implémentation, nous avons trouvé nécessaire d'arrondir les coefficients pour obtenir des coefficients nuls hors de la bidiagonale.

Montrons maintenant que l'invariant $Q_{left} \times BD \times Q_{right} = A$ est conservé à chaque tour de boucle. Pour cela, on raisonne par induction.

Initialisation :

On a initialement $Q_{left} = I_n, Q_{right} = I_n$ et $BD = A$. Il est donc clair que l'invariant est vrai au tour 0 de l'algorithme.

Induction :

Soit $k \in \mathbb{N}$ un entier quelconque fixé, on suppose l'invariant vérifié à ce rang : $Q_{left,k}BD_kQ_{right,k} = A$. Commençons par la multiplication à gauche. On construit une matrice de Householder Q de sorte que l'on obtienne la matrice $Q_{left,k+1} = Q_{left,k}Q$. En rappelant qu'à cette étape, $Q_{right,k+1} =$

$Q_{right,k}$ et $BD_{k+1} = QBD_k$, on peut vérifier l'invariant :

$$\begin{aligned} Q_{left,k+1}BD_{k+1}Q_{right,k+1} &= Q_{left,k}Q \cdot QBD_k \cdot Q_{right,k} \text{ or, } Q \text{ est de Householder, donc } Q^t = Q \text{ et } Q^2 = I_n \\ &= Q_{left,k}BD_kQ_{right,k} \\ &= A, \text{ par induction.} \end{aligned}$$

Pour ce qui est de la multiplication à droite par $Q_{right,k+1}$ une fois modifiée, il est clair que cette opération est symétrique à ce qui vient d'être fait précédemment.

Ce qui termine la preuve d'invariance de $Q_{left} \times BD \times Q_{right} = A$.

Transformation QR

Dans cette partie, nous avons implémenté un algorithme de transformation QR pour faire converger une matrice vers une matrice diagonale.

On constate sur l'image ?? que la convergence de l'algorithme subit deux phénomènes de cutoff, l'un à partir de 20 itérations, l'autre à partir de 45 itérations où la convergence devient plus rapide. Cependant, au delà de 95 itération, on observe que la convergence au contraire devient plus lente.

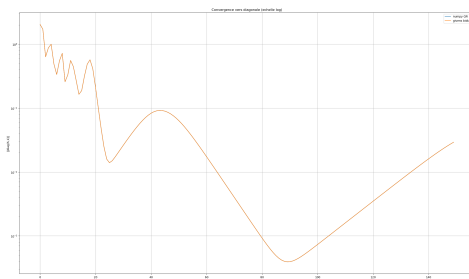


FIGURE 1 – Convergence de l'algorithme de transformation QR selon le nombre d'itérations.

Nous avons aussi vérifié que l'invariant se conserve à chaque tour de boucle. On constate, en effet que c'est le cas à l'aide de la figure ??. Où l'écart entre les valeurs de BD et de Q^tBDQ reste dans un intervalle acceptable.



FIGURE 2 – Vérification du variant.

Enfin, nous avons constaté que le temps d'exécution de l'algorithme de transformation QR est linéaire en fonction de la taille de la matrice. En effet, on observe une croissance linéaire du temps d'exécution selon la taille de la matrice comme le montre la figure ??. Avec, toutefois, un temps d'exécution plus élevé pour les matrices bidiagonales que pour les matrices pleine.

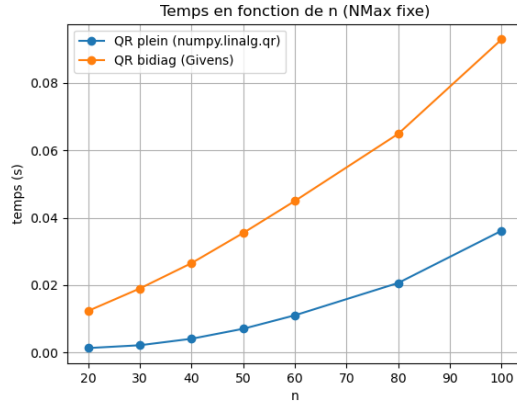


FIGURE 3 – Temps d'exécution de l'algorithme en fonction de la taille de la matrice.

Application à la compression d'images

Dans cette partie finale, nous allons appliquer l'algorithme de compression par factorisation SVD sur une image de test. Notre objectif est d'étudier la performance de cet algorithme. Pour cela, nous pourrions comparer sa performance à certains algorithmes optimisés de la librairie PIL. Voici nos différents critères pour évaluer la performance des algorithmes :

- L'appréciation visuelle, primordiale
- La taille de l'image obtenue
- Le PSNR de l'image compressée
- Le temps de calcul de la SVD

Rappel sur le PSNR

J'ai personnellement découvert le PSNR à travers mon TIPE en prépa qui portait sur le débruitage d'images par transformation en ondelettes. C'est un outil intéressant pour quantifier la qualité d'une image compressée ou modifiée. On le définit comme :

$$PSNR = 10 \log_{10} \frac{255^2}{MSE}$$

Ainsi, un PSNR de 40dB correspond à une erreur quadratique moyenne $MSE = \frac{255^2}{10^{\frac{40}{10}}} = 6.5$ soit une erreur inférieure à 1 % par pixel. Nous allons donc considérer par la suite qu'un PSNR supérieur à 40 dB décrira une qualité de compression satisfaisante.

Optimisation de l'algorithme

Avant de tester l'algorithme de compression, il est intéressant de voir quels vecteurs d'optimisation nous disposons pour accélérer l'algorithme. Une chose intéressante à faire est de voir la vitesse de convergence de l'algorithme `svd_qr` selon la valeur de `NMax`. Nous vous présentons le résultat suivant :

On remarquera toutefois que les valeurs de PSNR obtenues selon un `NMax` fixé dépendent de la valeur de k . Malgré cela, on décide qu'une valeur de `NMax` comprise entre 10 et 20 est suffisante. Par la suite, nous fixerons la valeur de `NMax` à 12 pour tous les tests.

Tests de performance

Appréciation visuelle

Ce premier test permet de mieux appréhender l'effet visuel de notre compression, en particulier l'influence du rang k . Toutefois, cela ne constitue pas un test factuel sur les performances. Voici quelques images compressées obtenues :

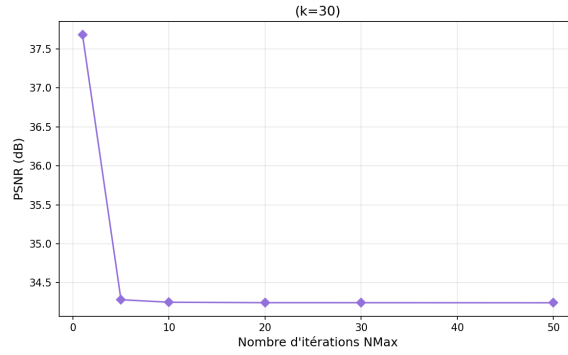


FIGURE 4 – Convergence de l'algorithme `svd_qr` selon la valeur de `NMax`.



FIGURE 5 – Images obtenues par compression pour des valeurs de `k` différentes.

Visuellement, nous pouvons déjà observer que plus `k` est élevé, plus l'image obtenue après compression est proche de l'image de base.

Influence du rang de compression sur le PSNR

Passons à des tests plus concrets. Ce deuxième test met en valeur l'influence du rang de compression sur le PSNR. Nous avons pris le soin de mettre en valeur la zone de résultats de bonne qualité.

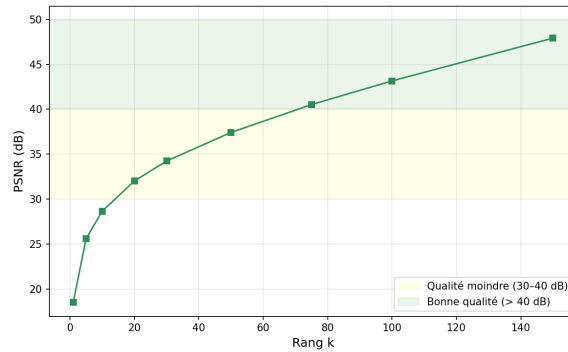


FIGURE 6 – PSNR selon les valeurs de rang de compression

Ce nouveau test nous permet de confirmer l'intuition que nous avons pu nous faire grâce au test précédent : une valeur de rang de compression élevée produit une image de meilleure qualité. Cependant, nous pouvons aussi attendre qu'une image de meilleure qualité produise un fichier de plus grande taille. Or, le but même de la compression est d'obtenir un fichier de taille inférieure. Ainsi, on se propose de voir l'influence du rang de compression sur la taille du fichier dans le test suivant.

Influence du rang de compression sur la taille du fichier

Nous avons donc implémenté un nouveau test mettant en concurrence la taille du fichier et le rang de compression de notre algorithme. Voici le résultat :

Remarque : pour le calcul de la taille du fichier, nous utilisons la librairie python `os`. Le résultat de ce test nous surprend. En effet, la taille du fichier compressé augmente peu (relativement à la taille

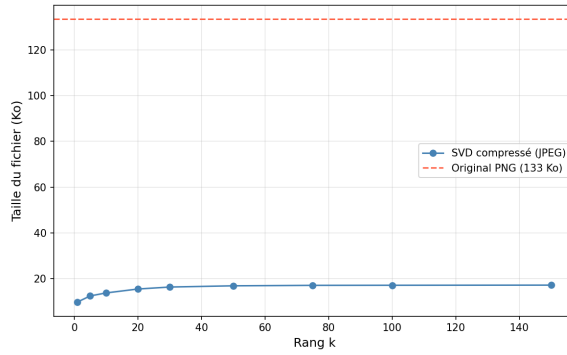


FIGURE 7 – Taille du fichier compressé selon le rang de compression

de l'image originale) avec l'augmentation du rang de compression.

Ainsi, ce résultat nous assure qu'il sera possible d'obtenir un PSNR satisfaisant tout en réduisant la taille du fichier de manière significative.

Influence du rang de compression sur le temps d'exécution

Le dernier point de test intéressant à aborder est le temps d'exécution de l'algorithme. Il faut cependant remarquer que les résultats suivants dépendent largement de la machine utilisée (ici un Ryzen 5 5500U). Voici un résultat obtenu :

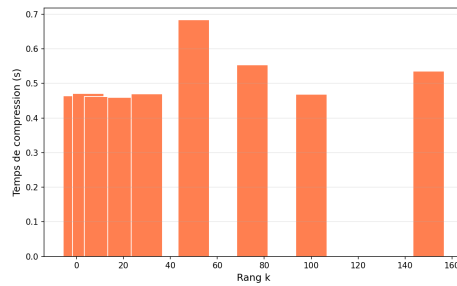


FIGURE 8 – Temps d'exécution de la compression selon le rang de compression

Ce test a des résultats qui semblent assez aléatoires, et l'on ne peut pas tirer de leçon claire concernant l'influence du rang sur le temps de compression. Par contre, il est clair que plus la valeur **NMax** est faible, plus le temps de compression l'est (ce qui est naturel étant donné la construction de l'algorithme `svd_qr`) :

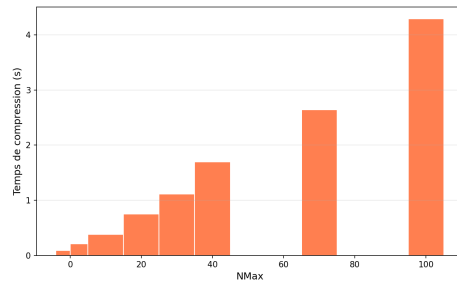


FIGURE 9 – Temps d'exécution de la compression selon la valeur de NMax

Conclusion sur les performance de la compression

Grâce à tous ces tests, nous pouvons affirmer qu'il est possible d'obtenir un compromis entre les valeurs de N_{Max} et de k pour obtenir un taux de compression satisfaisant sans perte visible de qualité. Cependant, nous devons nuancer les résultats de nos tests. En effet, nous n'avons testé l'algorithme avec une unique image test, ce qui ne nous permet pas d'identifier de possible biais lors de la compression.